

How to Run the Mental Wellness Chatbot Project

This guide explains how to set up the project, run both the backend and chatbot, and understand the purpose of each file.

1. Extract the Project Folder

1. Extract the ZIP file anywhere on your computer.

Example folder structure:

C:\MentalHealthProject\project-code

2. Before continuing, delete the following folders if they appear:

- backend\venv
- chatbot\venv
- Any folder named **pycache**

Each teammate must recreate their own virtual environments.

2. Install Python

Ensure that Python 3.10 or newer is installed.

Check the version:

```
python --version
```

If Python is missing, download it from:

<https://www.python.org/downloads/>

During installation, make sure to enable:

Add Python to PATH

3. Open the Project in VS Code

1. Open Visual Studio Code
2. Select: File → Open Folder
3. Choose the extracted folder:

C:\MentalHealthProject\project-code

4. Create Virtual Environments

The project has two independent components:

- **Backend (Flask server)**
- **Chatbot (Gradio interface)**

Each one must have **its own virtual environment**.

4.1 Create the Backend Environment

Open the VS Code terminal and switch to the backend folder:

```
cd backend
```

Create the virtual environment:

```
python -m venv venv
```

Activate it:

```
venv\Scripts\activate
```

You will now see:

```
(venv) C:\MentalHealthProject\project-code\backend>
```

4.2 Create the Chatbot Environment

Open a **new separate terminal** in VS Code.

Switch to the chatbot folder:

```
cd chatbot
```

Create the virtual environment:

```
python -m venv venv
```

Activate it:

```
venv\Scripts\activate
```

You will now see:

```
(venv) C:\MentalHealthProject\project-code\chatbot>
```

5. Install Required Packages

Backend Packages

Inside the backend terminal (the one where the venv is active):

```
pip install -r requirements.txt
```

If the backend has no requirements file, install manually:

```
pip install flask
```

```
pip install tenseal
```

```
pip install python-dotenv
```

```
pip install requests
```

Chatbot Packages

Inside the chatbot terminal (its own venv):

```
pip install -r requirements.txt
```

If no requirements file exists, install manually:

```
pip install gradio==4.44.0
```

```
pip install huggingface_hub
```

```
pip install python-dotenv
```

```
pip install requests
```

6. Configure the HuggingFace Token

Inside the **chatbot** folder, open the file:

```
chatbot.env
```

The file should contain:

```
HUGGINGFACE_API_TOKEN=your_token_here
```

```
HF_MODEL_ID=meta-llama/Llama-3.1-8B-Instruct
```

Notes:

- It is okay if all teammates use the same token **only if the owner allows it**.
- Otherwise, each teammate should generate their own HuggingFace API token.

7. Start the Backend Server

The backend handles:

- Receiving encrypted answers
- Performing homomorphic encryption computations
- Returning encrypted results to the chatbot

Using the backend terminal:

```
cd backend
venv\Scripts\activate
python server.py
```

You should see:

Server is running (HE backend)
Running on <http://127.0.0.1:5000>

Leave this terminal running.

8. Start the Chatbot Interface

Open the separate chatbot terminal.

Run:

```
cd chatbot
venv\Scripts\activate
python app.py
```

You will see:

Running on local URL: <http://127.0.0.1:7860>

9. Open the Chatbot in a Browser

Visit:

<http://127.0.0.1:7860>

You can now use:

- The **Chat** tab (LLM emotional support assistant)
- The **Private Stress Check** tab (fully encrypted HE scoring)

10. File Overview

Below is an explanation of the important files your teammates will see.

backend Folder

File / Folder	Purpose
server.py	Receives encrypted answers, performs the homomorphic weighted-sum computation, and returns encrypted scores.
encryption_module.py	Implements CKKS context creation, encryption, decryption, serialization, and deserialization. Shared logic between client and server.
requirements.txt	Python dependencies for backend.
test_client.py	Optional tool to test the backend manually. Not needed for running the chatbot. Can be deleted.
venv/	Should be deleted before sharing the project.
pycache/	Temporary auto-generated files. Safe to delete.

chatbot Folder

File / Folder	Purpose
app.py	Main Gradio UI. Handles chatbot logic, encrypted stress check, and calls the backend.
.env	Stores HuggingFace API token and model ID. Must be created or updated.
encryption_module.py	A clean copy of the backend encryption module, used client-side.
requirements.txt	Python dependencies for chatbot.
venv/	Delete before sharing. Each teammate creates their own.
pycache/	Can be deleted safely.

11. Files That Should Be Deleted Before Sharing the ZIP

File / Folder	Reason
backend\venv	Every teammate must recreate it.
chatbot\venv	Every teammate must recreate it.
Any pycache folder	Temporary files.